

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа № 2 по курсу
«Операционные системы и архитектура компьютера»**

Выполнил: А. А. Копылов

Группа: М8О-204БВ-25

Преподаватель: Н. А. Гапонов

Дата: 08.10.2026

Оценка: _____

Москва, 2026

Условие

Цель работы: приобрести практические навыки управления процессами и обмена данными между ними через каналы операционной системы.

Задание: разработать и отладить программу на языке C, создающую дочерние процессы и использующую каналы для их взаимодействия. Необходимо проверять результаты системных операций и освобождать выделенные ресурсы.

Вариант 12, группа 3. Родитель создаёт два дочерних процесса и принимает строки произвольной длины. Child1 переводит строки в верхний регистр, Child2 убирает все задвоенные пробелы. Родитель и дети представлены разными программами; результат Child2 возвращается родителю и выводится в стандартный поток вывода.

Метод решения

Созданы три однонаправленных неименованных канала: `input_pipe` от родителя к Child1, `middle_pipe` от Child1 к Child2 и `output_pipe` от Child2 к родителю. Родитель выполняет два `fork`. В каждом ребёнке `dup2` связывает соответствующие концы каналов со стандартными вводом и выводом, затем `exec1` запускает отдельный исполняемый файл.

Схема взаимодействия процессов

```
parent -> input_pipe -> child1 -> middle_pipe -> child2 -> output_pipe -> parent
```

Каждый процесс закрывает неиспользуемые дескрипторы. Данные разделены переводом строки. `getline` увеличивает динамический буфер по мере необходимости. Первый ребёнок применяет `toupper` к байтам ASCII. Второй сохраняет первый обычный пробел в каждой последовательности и пропускает последующие; табуляции и одиночные начальные или конечные пробелы сохраняются.

Родитель переводит рабочие концы каналов в неблокирующий режим и использует `poll` для продвижения записи и чтения. Он учитывает частичную запись и ждёт конца ответной строки, а не EOF после каждого ответа. При EOF пользовательского ввода родитель закрывает канал, дети завершают чтение, а `waitpid` собирает их статусы. PID и сообщения об ошибках идут в `stderr`, чтобы не смешиваться с передаваемыми строками.

Для справки использованы Linux man-pages:
<https://man7.org/linux/man-pages/man2/pipe.2.html> ;
<https://man7.org/linux/man-pages/man2/dup.2.html> ;
<https://man7.org/linux/man-pages/man1/strace.1.html> .

Описание программы

Структура файлов

Таблица 1 — Файлы проекта

Файл	Назначение
parent.c	Создание каналов и детей, перенаправление потоков, обмен и ожидание завершения.
child1.c	Чтение строк и перевод ASCII-букв в верхний регистр.
child2.c	Сжатие последовательностей обычных пробелов в том же буфере.
common.c, common.h	Проверяемые <code>close</code> и <code>dup2</code> , полная запись <code>write_all</code> .

Файл	Назначение
CMakeLists.txt	Сборка трёх исполняемых файлов и общего модуля, запуск тестов.
tests/test_pipeline.sh	Проверка обычного ввода, пустой строки, EOF без перевода строки, длинных строк и ошибки запуска ребёнка.
tools/capture-linux.sh	Сборка и получение воспроизводимых журналов и полного strace в Linux.
tools/test-errors.sh	Инъекция ошибок pipe/fork и проверка нехватки памяти при getline.
tools/Dockerfile	Ubuntu 24.04 с GCC, CMake и strace для запуска на Mac через Docker Desktop.

Основные типы данных и функции

Каналы представлены массивами `int[2]`: индекс 0 означает конец чтения, индекс 1 — конец записи. `pid_t` хранит результаты `fork` и идентификаторы детей. `size_t` используется для размера буфера и количества отправленных байтов; `ssize_t` допускает значение -1 при ошибке чтения или записи. `struct pollfd` содержит дескриптор, запрошенные события и события, возвращённые `poll`.

`child1_process` и `child2_process` настраивают стандартные потоки и выполняют `execl`. `close_pipe` и `close_three_pipes` закрывают дескрипторы, записывая -1 вместо закрытого номера. `exchange_line` принимает два дескриптора, указатель и длину строки, продвигает передачу до отправки всех байтов и получения ответного перевода строки; возвращает 0 при успехе и -1 при ошибке. `wait_for_child` получает статус ребёнка и проверяет код завершения. `write_all` повторяет частичные записи и `EINTR`.

Используемые системные вызовы

Таблица 2 — Операции интерфейса ОС

Системный вызов	Назначение в программе
<code>pipe</code>	Создание трёх каналов; в полученной Linux-трассе соответствует <code>pipe2</code> .
<code>fork</code>	Создание двух детей; в Linux библиотечная реализация вызывает <code>clone</code> .
<code>dup2</code>	Перенаправление <code>stdin</code> и <code>stdout</code> детей; в Ubuntu ARM64 наблюдается <code>dup3</code> .
<code>execl</code> / <code>execve</code>	Замена программы ребёнка на <code>child1</code> или <code>child2</code> без изменения PID.
<code>read</code> , <code>write</code>	Чтение результата и передача байтов между процессами.
<code>fcntl</code>	Получение флагов и установка <code>O_NONBLOCK</code> на рабочих концах родителя.
<code>poll</code>	Ожидание готовности чтения и записи; в ARM64 трассе виден <code>ppoll</code> .
<code>close</code>	Закрывание лишних и использованных дескрипторов; распространение EOF.

Системный вызов	Назначение в программе
waitpid / wait4	Ожидание детей, получение статусов и удаление записей зомби.
signal / rt_sigaction	Игнорирование SIGPIPE; запись в разорванный канал обрабатывается как ошибка.
getpid, _exit	Получение PID для диагностики и завершение ребёнка при неудачной подготовке или exec.

getline, toupper, malloc/realloc, free, fprintf и perror — библиотечные функции. Они не являются отдельными одноимёнными системными вызовами. В полном strace также видны загрузка libc, отображение памяти и завершение процессов, необходимые среде выполнения.

Ключевые фрагменты кода

Перенаправление первого ребёнка выполняется до exec. В случае ошибки ребёнок завершается через _exit; при успехе execl управление не возвращается.

Листинг 1 — Подготовка Child1 (parent.c)

```
static void child1_process(int input_pipe[2], int middle_pipe[2],
                           int output_pipe[2], const char *child1_path) {
    if (dup2_checked(input_pipe[READ_END], STDIN_FILENO, "child1") == -1 ||
        dup2_checked(middle_pipe[WRITE_END], STDOUT_FILENO, "child1") == -1) {
        _exit(126);
    }

    if (close_three_pipes(input_pipe, middle_pipe, output_pipe, "child1") == -1) {
        _exit(126);
    }

    execl(child1_path, child1_path, (char *) NULL);
    perror("parent: exec child1");
    _exit(127);
}
```

Запись должна учитывать фактическое число переданных байтов; при EINTR она повторяется.

Листинг 2 — Полная запись (common.c)

```
ssize_t write_all(int fd, const void *buffer, size_t count) {
    const unsigned char *data = buffer;
    size_t written = 0;

    while (written < count) {
        ssize_t result = write(fd, data + written, count - written);
        if (result > 0) {
            written += (size_t) result;
            continue;
        }
        if (result == -1 && errno == EINTR) {
            continue;
        }
        return -1;
    }

    return (ssize_t) written;
}
```

Родитель одновременно отслеживает готовность каналов. Таймаут -1 означает ожидание события; EINTR не считается окончательной ошибкой.

Листинг 3 — Ожидание готовности (parent.c)

```
while (sent < total || !received_newline) {
    struct pollfd fds[2] = {
        { .fd = input_fd, .events = sent < total ? POLLOUT : 0 },
        { .fd = output_fd, .events = received_newline ? 0 : POLLIN }
    };

    int ready;
    do {
        ready = poll(fds, 2, -1);
    } while (ready == -1 && errno == EINTR);
    if (ready == -1) {
        perror("parent: poll");
        return -1;
    }
}
```

Результаты

Проверка выполнена 08.10.2026 на Mac через Docker Desktop, в контейнере Ubuntu 24.04 ARM64. Проект заново собран Linux-компилятором. Ниже приведены сведения о среде, результат запуска и журналы проверки.

Среда выполнения

```
Linux 83d944d35fb1 6.12.76-linuxkit #1 SMP Sat Jul 11 11:18:11 UTC 2026 aarch64 aarch64 aarch64 GNU/Linux
PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.5 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
cc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

При демонстрации обработаны две строки подряд. Диагностика фиксирует PID всех трёх процессов. Начальные и конечные серии пробелов сокращаются до одного пробела.

Запуск, входные данные, результат и PID

```
$ strace -f -s 256 -o report/evidence/strace.txt ./build-linux/parent ./build-linux/child1 ./build-linux/child2 <
report/evidence/input.txt
Input:
hello world
 a b
Output:
HELLO WORLD
A B
Process diagnostics:
[parent pid=168, child1=169, child2=170] started
```

```

Enter lines. Press Ctrl-D to finish.
[child2 pid=170] started
[child1 pid=169] started
[child1 pid=169] finished
[child2 pid=170] finished
[parent pid=168] finished
Tests:
Internal ctest changing into directory: /lab/build-linux
Test project /lab/build-linux
  Start 1: pipeline_tests
1/1 Test #1: pipeline_tests ..... Passed   0.06 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =  0.07 sec

```

Автоматическая проверка покрывает обычную строку, пустую строку, отсутствие повторных пробелов, начальные и конечные пробелы, последнюю строку без перевода строки, строку со 100 000 пробелов, строку из 200 000 букв и невозможность запуска Child1. Ограничение времени теста — 30 секунд.

Ошибки создания канала и детей проверены инъекцией strace. Нехватка памяти при getline проверена ограничением виртуальной памяти процесса до 32 МиБ и более длинным входным потоком. Во всех аварийных запусках программа вывела причину ошибки и завершилась с кодом 1 без зависания. Это отдельные диагностические запуски; успешная демонстрационная трасса в приложении Б не содержит инъекций.

Проверка обработки ошибок системных вызовов

```

inject=pipe2:error=EMFILE:when=1; exit=1; no timeout
parent: pipe input: Too many open files
[parent pid=17] finished
inject=clone:error=EAGAIN:when=1; exit=1; no timeout
parent: fork child1: Resource temporarily unavailable
[parent pid=26] finished
inject=clone:error=EAGAIN:when=2; exit=1; no timeout
parent: fork child2: Resource temporarily unavailable
[child1 pid=36] started
[child1 pid=36] finished
[parent pid=35] finished
getline allocation failure: virtual-memory limit 32 768 KiB; exit=1; no timeout
[parent pid=40, child1=41, child2=42] started
Enter lines. Press Ctrl-D to finish.
[child1 pid=41] started
[child2 pid=42] started
parent: getline: Cannot allocate memory
[child1 pid=41] finished
[child2 pid=42] finished
[parent pid=40] finished

```

Ограничение преобразования: регистр обрабатывается побайтово для ASCII, поэтому русский UTF-8 не переводится в верхний регистр. Проверки использовали латинский текст. Строки не ограничены фиксированным массивом, однако получение памяти может завершиться ошибкой; результат getline проверяется, буфер освобождается.

Демонстрация работы strace

Запуск: `strace -f -s 256 -o report/evidence/strace.txt ./build-linux/parent ./build-linux/child1 ./build-linux/child2 < report/evidence/input.txt`. Ключ `-f` отслеживает

обоих детей, -o сохраняет журнал. Фильтр -e trace не применялся: приложение Б содержит все строки сохранённой трассы. Параметр -s задаёт предел представления строковых аргументов, а не удаляет системные вызовы.

Создание каналов и процессов

```
168 pipe2([3, 4], 0) = 0
168 pipe2([5, 6], 0) = 0
168 pipe2([7, 8], 0) = 0
168 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffff88c54fb0) = 169
168 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD <unfinished ...>
168 <... clone resumed>, child_tidptr=0xffff88c54fb0) = 170
```

Три pipe2 создают пары концов чтения и записи. Два clone соответствуют fork; положительные возвращённые значения — PID детей.

Перенаправление и запуск отдельных программ

```
169 dup3(3, 0, 0 <unfinished ...>
169 <... dup3 resumed>) = 0
170 dup3(5, 0, 0 <unfinished ...>
169 dup3(6, 1, 0 <unfinished ...>
170 <... dup3 resumed>) = 0
169 <... dup3 resumed>) = 1
170 dup3(8, 1, 0 <unfinished ...>
170 <... dup3 resumed>) = 1
169 execve("./build-linux/child1", ["/build-linux/child1"], 0xffffffff234328 /* 5 vars */ <unfinished ...>
170 execve("./build-linux/child2", ["/build-linux/child2"], 0xffffffff234328 /* 5 vars */ <unfinished ...>
169 <... execve resumed>) = 0
170 <... execve resumed>) = 0
```

dup3 подключает каналы к дескрипторам 0 и 1. execve запускает child1 и child2; PID каждого ребёнка сохраняется.

Передача строк

```
168 write(4, "hello world\n", 14) = 14
169 read(0, "hello world\n", 4096) = 14
169 write(1, "HELLO WORLD\n", 14) = 14
170 write(1, "HELLO WORLD\n", 12 <unfinished ...>
170 <... write resumed>) = 12
168 write(1, "HELLO WORLD\n", 12) = 12
170 write(1, " A B \n", 6 <unfinished ...>
170 <... write resumed>) = 6
168 write(1, " A B \n", 6) = 6
```

В write и read видны исходная строка, строка в верхнем регистре и результат с одним пробелом. Возвращённое число означает реально переданные байты.

Неблокирующий режим и ожидание готовности

```
168 fcntl(4, F_GETFL <unfinished ...>
168 <... fcntl resumed>) = 0x1 (flags O_WRONLY)
168 fcntl(4, F_SETFL, O_WRONLY|O_NONBLOCK <unfinished ...>
168 <... fcntl resumed>) = 0
168 fcntl(7, F_GETFL <unfinished ...>
168 <... fcntl resumed>) = 0 (flags O_RDONLY)
168 fcntl(7, F_SETFL, O_RDONLY|O_NONBLOCK <unfinished ...>
168 <... fcntl resumed>) = 0
168 ppoll([{fd=4, events=POLLOUT}, {fd=7, events=POLLIN}], 2, NULL, NULL, 0 <unfinished ...>
168 <... ppoll resumed>) = 1 ({fd=4, events=POLLOUT})
168 ppoll([{fd=4, events=0}, {fd=7, events=POLLIN}], 2, NULL, NULL, 0 <unfinished ...>
```

```

168 <... ppoll resumed>          = 1 ({fd=7, revents=POLLIN})
168 ppoll([{fd=4, events=POLLOUT}, {fd=7, events=POLLIN}], 2, NULL, NULL, 0) = 1 ({fd=4, revents=POLLOUT})
168 ppoll([{fd=4, events=0}, {fd=7, events=POLLIN}], 2, NULL, NULL, 0 <unfinished ...>
168 <... ppoll resumed>          = 1 ({fd=7, revents=POLLIN})

```

fcntl устанавливает O_NONBLOCK. ppoll ожидает POLLIN/POLLOUT: сам вызов не переносит данные.

EOF и ожидание завершения

```

168 read(0, "", 4096)          = 0
169 <... read resumed> "", 4096) = 0
168 wait4(169, <unfinished ...>
170 <... read resumed> "", 4096) = 0
169 +++ exited with 0 +++
168 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 169
168 wait4(170, <unfinished ...>
170 +++ exited with 0 +++
168 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 170

```

read с результатом 0 показывает конец входного потока у родителя и EOF в каналах у детей. wait4 забирает их статусы; нормальное завершение имеет код 0.

Строки <unfinished ...> и <... resumed> означают, что системный вызов одного процесса начался, но завершился после событий другого. Случайный порядок сообщений о запуске допустим: выполнение процессов планируется ОС.

Выводы

Реализован обмен строками между родителем и двумя отдельными дочерними программами через три канала POSIX. Освоены создание процессов, перенаправление стандартных потоков, проверка системных ошибок и получение статусов завершения. Проверки в Ubuntu подтвердили преобразование строк и обработку краевых случаев; трасса связала действия программы с реальными вызовами ядра. Неиспользуемые концы каналов закрываются, память освобождается, завершившиеся дети собираются через waitpid. Для завершения ввода необходимо закрыть запись до ожидания детей, иначе читатели могли бы ожидать EOF, а родитель — их завершение.

Приложение А. Исходный код программы

Листинг А.1 — Файл src/parent.c

```
#define _POSIX_C_SOURCE 200809L

#include "common.h"

#include <errno.h>
#include <fcntl.h>
#include <poll.h>
#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

enum { READ_END = 0, WRITE_END = 1 };

static int close_pipe(int pipe_fd[2], const char *name) {
    int status = 0;
    if (close_checked(pipe_fd[READ_END], name) == -1) {
        status = -1;
    }
    pipe_fd[READ_END] = -1;
    if (close_checked(pipe_fd[WRITE_END], name) == -1) {
        status = -1;
    }
    pipe_fd[WRITE_END] = -1;
    return status;
}

static int close_three_pipes(int first[2], int second[2], int third[2],
                             const char *name) {
    int status = 0;
    if (close_pipe(first, name) == -1) {
        status = -1;
    }
    if (close_pipe(second, name) == -1) {
        status = -1;
    }
    if (close_pipe(third, name) == -1) {
        status = -1;
    }
    return status;
}

static void child1_process(int input_pipe[2], int middle_pipe[2],
                           int output_pipe[2], const char *child1_path) {
    if (dup2_checked(input_pipe[READ_END], STDIN_FILENO, "child1") == -1 ||
        dup2_checked(middle_pipe[WRITE_END], STDOUT_FILENO, "child1") == -1) {
        _exit(126);
    }

    if (close_three_pipes(input_pipe, middle_pipe, output_pipe, "child1") == -1) {
        _exit(126);
    }

    execl(child1_path, child1_path, (char *) NULL);
```

```

    perror("parent: exec child1");
    _exit(127);
}

static void child2_process(int input_pipe[2], int middle_pipe[2],
                           int output_pipe[2], const char *child2_path) {
    if (dup2_checked(middle_pipe[READ_END], STDIN_FILENO, "child2") == -1 ||
        dup2_checked(output_pipe[WRITE_END], STDOUT_FILENO, "child2") == -1) {
        _exit(126);
    }

    if (close_three_pipes(input_pipe, middle_pipe, output_pipe, "child2") == -1) {
        _exit(126);
    }

    execl(child2_path, child2_path, (char *) NULL);
    perror("parent: exec child2");
    _exit(127);
}

static int wait_for_child(pid_t pid, const char *name) {
    int child_status;
    pid_t result;

    do {
        result = waitpid(pid, &child_status, 0);
    } while (result == -1 && errno == EINTR);

    if (result == -1) {
        fprintf(stderr, "parent: waitpid %s: ", name);
        perror(NULL);
        return -1;
    }

    if (WIFEXITED(child_status) && WEXITSTATUS(child_status) == 0) {
        return 0;
    }

    if (WIFEXITED(child_status)) {
        fprintf(stderr, "parent: %s exited with code %d\n",
                name, WEXITSTATUS(child_status));
    } else if (WIFSIGNALED(child_status)) {
        fprintf(stderr, "parent: %s terminated by signal %d\n",
                name, WTERMSIG(child_status));
    }

    return -1;
}

static int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL);
    if (flags == -1 || fcntl(fd, F_SETFL, flags | O_NONBLOCK) == -1) {
        perror("parent: fcntl");
        return -1;
    }

    return 0;
}

static int exchange_line(int input_fd, int output_fd, const char *line,
                          size_t length, bool add_newline) {
    size_t sent = 0;
    size_t total = length + (add_newline ? 1u : 0u);
    bool received_newline = false;

    while (sent < total || !received_newline) {

```

```

struct pollfd fds[2] = {
    { .fd = input_fd, .events = sent < total ? POLLOUT : 0 },
    { .fd = output_fd, .events = received_newline ? 0 : POLLIN }
};

int ready;
do {
    ready = poll(fds, 2, -1);
} while (ready == -1 && errno == EINTR);
if (ready == -1) {
    perror("parent: poll");
    return -1;
}

if (fds[1].revents & (POLLIN | POLLHUP | POLLERR)) {
    char buffer[4096];
    ssize_t count = read(output_fd, buffer, sizeof(buffer));
    if (count == 0) {
        fprintf(stderr, "parent: result pipe closed unexpectedly\n");
        return -1;
    }
    if (count == -1 && errno != EAGAIN && errno != EWOULDBLOCK && errno != EINTR) {
        perror("parent: read result");
        return -1;
    }
    if (count > 0) {
        for (ssize_t i = 0; i < count; ++i) {
            if (buffer[i] == '\n') {
                received_newline = true;
                if (i != count - 1) {
                    fprintf(stderr, "parent: unexpected extra output\n");
                    return -1;
                }
            }
        }
    }
    if (write_all(STDOUT_FILENO, buffer, (size_t) count) == -1) {
        perror("parent: write stdout");
        return -1;
    }
}

if (fds[0].revents & POLLOUT) {
    const char *data = sent < length ? line + sent : "\n";
    size_t remaining = sent < length ? length - sent : 1;
    ssize_t count = write(input_fd, data, remaining);
    if (count > 0) {
        sent += (size_t) count;
    } else if (count == -1 && errno != EAGAIN && errno != EWOULDBLOCK && errno != EINTR) {
        perror("parent: write input pipe");
        return -1;
    }
}

if (fds[0].revents & (POLLERR | POLLHUP | POLLNVAL)) {
    fprintf(stderr, "parent: input pipe closed unexpectedly\n");
    return -1;
}

if (fds[1].revents & POLLNVAL) {
    fprintf(stderr, "parent: invalid result pipe descriptor\n");
    return -1;
}

```

```

    }
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s PATH_TO_CHILD1 PATH_TO_CHILD2\n", argv[0]);
        return EXIT_FAILURE;
    }

    if (signal(SIGPIPE, SIG_IGN) == SIG_ERR) {
        perror("parent: signal");
        return EXIT_FAILURE;
    }

    int input_pipe[2] = {-1, -1};
    int middle_pipe[2] = {-1, -1};
    int output_pipe[2] = {-1, -1};
    pid_t child1_pid = -1;
    pid_t child2_pid = -1;
    int status = EXIT_FAILURE;

    if (pipe(input_pipe) == -1) {
        perror("parent: pipe input");
        goto cleanup;
    }
    if (pipe(middle_pipe) == -1) {
        perror("parent: pipe middle");
        goto cleanup;
    }
    if (pipe(output_pipe) == -1) {
        perror("parent: pipe output");
        goto cleanup;
    }

    child1_pid = fork();
    if (child1_pid == -1) {
        perror("parent: fork child1");
        goto cleanup;
    }
    if (child1_pid == 0) {
        child1_process(input_pipe, middle_pipe, output_pipe, argv[1]);
    }

    child2_pid = fork();
    if (child2_pid == -1) {
        perror("parent: fork child2");
        goto cleanup;
    }
    if (child2_pid == 0) {
        child2_process(input_pipe, middle_pipe, output_pipe, argv[2]);
    }

    int close_status = 0;
    if (close_checked(input_pipe[READ_END], "parent") == -1) {
        close_status = -1;
    }
    input_pipe[READ_END] = -1;
    if (close_checked(middle_pipe[READ_END], "parent") == -1) {
        close_status = -1;
    }
    middle_pipe[READ_END] = -1;

```

```

if (close_checked(middle_pipe[WRITE_END], "parent") == -1) {
    close_status = -1;
}
middle_pipe[WRITE_END] = -1;
if (close_checked(output_pipe[WRITE_END], "parent") == -1) {
    close_status = -1;
}
output_pipe[WRITE_END] = -1;
if (close_status == -1) {
    goto cleanup;
}
if (set_nonblocking(input_pipe[WRITE_END]) == -1 ||
    set_nonblocking(output_pipe[READ_END]) == -1) {
    goto cleanup;
}

fprintf(stderr, "[parent pid=%ld, child1=%ld, child2=%ld] started\n",
        (long) getpid(), (long) child1_pid, (long) child2_pid);
fprintf(stderr, "Enter lines. Press Ctrl-D to finish.\n");

char *line = NULL;
size_t capacity = 0;
for (;;) {
    errno = 0;
    ssize_t length = getline(&line, &capacity, stdin);
    if (length == -1) {
        if (feof(stdin)) {
            status = EXIT_SUCCESS;
        } else {
            perror("parent: getline");
        }
        break;
    }

    if (exchange_line(input_pipe[WRITE_END], output_pipe[READ_END],
        line, (size_t) length, length == 0 || line[length - 1] != '\n') == -1) {
        break;
    }
}
free(line);

cleanup:
if (close_three_pipes(input_pipe, middle_pipe, output_pipe, "parent") == -1) {
    status = EXIT_FAILURE;
}

int children_ok = 0;
if (child1_pid > 0 && wait_for_child(child1_pid, "child1") == -1) {
    children_ok = -1;
}
if (child2_pid > 0 && wait_for_child(child2_pid, "child2") == -1) {
    children_ok = -1;
}
if (children_ok == -1) {
    status = EXIT_FAILURE;
}

fprintf(stderr, "[parent pid=%ld] finished\n", (long) getpid());
return status;
}

```

Листинг А.2 — Файл src/child1.c

```
#define _POSIX_C_SOURCE 200809L

#include "common.h"

#include <ctype.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    char *line = NULL;
    size_t capacity = 0;
    int status = EXIT_SUCCESS;

    fprintf(stderr, "[child1 pid=%ld] started\n", (long) getpid());

    for (;;) {
        errno = 0;
        ssize_t length = getline(&line, &capacity, stdin);
        if (length == -1) {
            if (feof(stdin)) {
                break;
            }
            perror("child1: getline");
            status = EXIT_FAILURE;
            break;
        }

        for (ssize_t i = 0; i < length; ++i) {
            line[i] = (char) toupper((unsigned char) line[i]);
        }

        if (write_all(STDOUT_FILENO, line, (size_t) length) == -1) {
            perror("child1: write");
            status = EXIT_FAILURE;
            break;
        }
    }

    free(line);
    if (fclose(stdin) == EOF) {
        perror("child1: fclose stdin");
        status = EXIT_FAILURE;
    }
    if (close_checked(STDOUT_FILENO, "child1") == -1) {
        status = EXIT_FAILURE;
    }

    fprintf(stderr, "[child1 pid=%ld] finished\n", (long) getpid());
    return status;
}
```

Листинг А.3 — Файл src/child2.c

```
#define _POSIX_C_SOURCE 200809L

#include "common.h"

#include <errno.h>
```

```

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    char *line = NULL;
    size_t capacity = 0;
    int status = EXIT_SUCCESS;

    fprintf(stderr, "[child2 pid=%ld] started\n", (long) getpid());

    for (;;) {
        errno = 0;
        ssize_t length = getline(&line, &capacity, stdin);
        if (length == -1) {
            if (feof(stdin)) {
                break;
            }
            perror("child2: getline");
            status = EXIT_FAILURE;
            break;
        }

        bool previous_was_space = false;
        size_t output_length = 0;
        for (ssize_t i = 0; i < length; ++i) {
            if (line[i] == ' ') {
                if (previous_was_space) {
                    continue;
                }
                previous_was_space = true;
            } else {
                previous_was_space = false;
            }

            line[output_length++] = line[i];
        }
        if (write_all(STDOUT_FILENO, line, output_length) == -1) {
            perror("child2: write");
            status = EXIT_FAILURE;
            break;
        }
    }

    free(line);
    if (fclose(stdin) == EOF) {
        perror("child2: fclose stdin");
        status = EXIT_FAILURE;
    }
    if (close_checked(STDOUT_FILENO, "child2") == -1) {
        status = EXIT_FAILURE;
    }

    fprintf(stderr, "[child2 pid=%ld] finished\n", (long) getpid());
    return status;
}

```

Листинг А.4 — Файл src/common.c

```

#include "common.h"

```

```

#include <errno.h>
#include <stdio.h>
#include <unistd.h>

int close_checked(int fd, const char *name) {
    if (fd < 0) {
        return 0;
    }

    if (close(fd) == -1) {
        fprintf(stderr, "%s: close: ", name);
        perror(NULL);
        return -1;
    }
    return 0;
}

int dup2_checked(int old_fd, int new_fd, const char *name) {
    if (dup2(old_fd, new_fd) == -1) {
        fprintf(stderr, "%s: dup2: ", name);
        perror(NULL);
        return -1;
    }
    return 0;
}

ssize_t write_all(int fd, const void *buffer, size_t count) {
    const unsigned char *data = buffer;
    size_t written = 0;

    while (written < count) {
        ssize_t result = write(fd, data + written, count - written);
        if (result > 0) {
            written += (size_t) result;
            continue;
        }
        if (result == -1 && errno == EINTR) {
            continue;
        }
        return -1;
    }

    return (ssize_t) written;
}

```

Листинг А.5 — Файл src/common.h

```

#ifndef LAB2_COMMON_H
#define LAB2_COMMON_H

#include <stddef.h>
#include <sys/types.h>

int close_checked(int fd, const char *name);
int dup2_checked(int old_fd, int new_fd, const char *name);
ssize_t write_all(int fd, const void *buffer, size_t count);

#endif

```

Листинг А.6 — Файл src/CMakeLists.txt

```

cmake_minimum_required(VERSION 3.16)

```

```

project(lab2_variant12 C)
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS ON)

add_library(common STATIC common.c)
target_include_directories(common PUBLIC ${CMAKE_CURRENT_SOURCE_DIR})
target_compile_options(common PRIVATE -Wall -Wextra -Wpedantic)
foreach(program parent child1 child2)
    add_executable(${program} ${program}.c)
    target_link_libraries(${program} PRIVATE common)
    target_compile_options(${program} PRIVATE -Wall -Wextra -Wpedantic)
endforeach()

enable_testing()
add_test(NAME pipeline_tests
    COMMAND sh ${CMAKE_CURRENT_SOURCE_DIR}/tests/test_pipeline.sh
        ${TARGET_FILE:parent} ${TARGET_FILE:child1} ${TARGET_FILE:child2})
set_tests_properties(pipeline_tests PROPERTIES TIMEOUT 30)

```

Листинг А.7 — Файл src/tests/test_pipeline.sh

```

#!/bin/sh
set -eu

parent=$1
child1=$2
child2=$3
temporary_directory=$(mktemp -d "${TMPDIR:-/tmp}/lab2-test.XXXXXX")
trap 'rm -rf "$temporary_directory"' EXIT HUP INT TERM

run_case() {
    name=$1
    input=$2
    expected=$3

    printf '%s' "$input" | "$parent" "$child1" "$child2" \
        >"$temporary_directory/actual" 2>"$temporary_directory/diagnostics"
    printf '%s' "$expected" >"$temporary_directory/expected"

    if ! cmp -s "$temporary_directory/expected" "$temporary_directory/actual"; then
        echo "FAILED: $name" >&2
        diff -u "$temporary_directory/expected" "$temporary_directory/actual" >&2 || true
        exit 1
    fi
    echo "PASSED: $name"
}

run_case "uppercase and spaces" \
    'Hello  world'
    '\
    'HELLO WORLD'
    '

run_case "leading and trailing spaces" \
    ' one  two'
    '\
    ' ONE TWO
    '

run_case "empty line" '
''

```

```

run_case "no duplicate spaces" \
    'Already_ok 123!'
'\
    'ALREADY_OK 123!
,

run_case "last line without newline" \
    'last line' \
    'LAST LINE
,

long_input=$temporary_directory/long_input
long_expected=$temporary_directory/long_expected
awk 'BEGIN { printf "a"; for (i = 0; i < 100 000; ++i) printf " "; print "b" }' >"$long_input"
printf 'A B\n' >"$long_expected"
"$parent" "$child1" "$child2" <"$long_input" \
    >"$temporary_directory/actual" 2>"$temporary_directory/diagnostics"
cmp "$long_expected" "$temporary_directory/actual"
echo "PASSED: long line"

awk 'BEGIN { for (i = 0; i < 100 000; ++i) printf "ab"; print "" }' >"$long_input"
awk 'BEGIN { for (i = 0; i < 100 000; ++i) printf "AB"; print "" }' >"$long_expected"
"$parent" "$child1" "$child2" <"$long_input" \
    >"$temporary_directory/actual" 2>"$temporary_directory/diagnostics"
cmp "$long_expected" "$temporary_directory/actual"
echo "PASSED: long line without spaces"

if printf 'test\n' | "$parent" /path/that/does/not/exist "$child2" \
    >"$temporary_directory/actual" 2>"$temporary_directory/diagnostics"; then
    echo "FAILED: broken child must cause a non-zero exit status" >&2
    exit 1
fi
echo "PASSED: broken child connection"

```

Листинг A.8 — Файл src/tools/capture-linux.sh

```

#!/bin/sh
# Run from any directory inside a Linux environment with CMake, C compiler, strace.
set -eu
task_root=$(CDPATH= cd -- "$(dirname -- "$0")/.." && pwd)
cd "$task_root"
if [ "$(uname -s)" != Linux ]; then
    echo 'Run this script in Linux (Ubuntu). macOS cannot provide this strace.' >&2
    exit 1
fi
for tool in cmake strace; do
    command -v "$tool" >/dev/null 2>&1 || {
        echo "Missing $tool. In Ubuntu: sudo apt install build-essential cmake strace" >&2
        exit 1
    }
done
mkdir -p report/evidence
{
    uname -a
    cat /etc/os-release
    cc --version
    cmake --version
    strace --version
} > report/evidence/environment.txt 2>&1
cmake -S src -B build-linux -DCMAKE_BUILD_TYPE=Debug > report/evidence/build.txt 2>&1

```

```

cmake --build build-linux --parallel >> report/evidence/build.txt 2>&1
ctest --test-dir build-linux --output-on-failure > report/evidence/tests.txt 2>&1
printf 'hello world\n a b \n' > report/evidence/input.txt
strace -f -s 256 -o report/evidence/strace.txt \
    ./build-linux/parent ./build-linux/child1 ./build-linux/child2 \
    < report/evidence/input.txt \
    > report/evidence/output.txt 2> report/evidence/processes.txt
printf 'HELLO WORLD\n A B \n' > report/evidence/expected.txt
cmp report/evidence/expected.txt report/evidence/output.txt
{
    echo '$ strace -f -s 256 -o report/evidence/strace.txt ./build-linux/parent ./build-linux/child1 ./build-linux/child2 <
report/evidence/input.txt'
    echo 'Input:'
    cat report/evidence/input.txt
    echo 'Output:'
    cat report/evidence/output.txt
    echo 'Process diagnostics:'
    cat report/evidence/processes.txt
    echo 'Tests:'
    cat report/evidence/tests.txt
} > report/evidence/demo.txt
# Saving an extract does not alter the complete trace used in report appendix B.
grep -E 'pipe2?\\(|clone3?\\(|fork\\(|dup[23]\\(|execve\\(|read\\(|write\\(|close\\(|wait4\\(|poll\\(|ppoll\\(|fcntl\\(|' \
    report/evidence/strace.txt > report/evidence/strace-extract.txt || true
echo 'Linux build, tests and real full strace saved in report/evidence/'

```

Листинг А.9 — Файл src/tools/test-errors.sh

```

#!/bin/sh
set -eu
task_root=$(CDPATH= cd -- "$(dirname -- "$0")/.." && pwd)
cd "$task_root"
mkdir -p report/evidence
: > report/evidence/faults.txt
for specification in pipe2:error=EMFILE:when=1 clone:error=EAGAIN:when=1 clone:error=EAGAIN:when=2; do
    label=$(printf '%s' "$specification" | tr ':= ' '_')
    if timeout 15 strace -f -e "inject=$specification" \
        -o "report/evidence/fault-$label.trace" \
        ./build-linux/parent ./build-linux/child1 ./build-linux/child2 \
        < report/evidence/input.txt > "report/evidence/fault-$label.out" \
        2> "report/evidence/fault-$label.err"; then
        echo "Expected failure not detected: $specification" >&2
        exit 1
    else
        result=$?
        [ "$result" -eq 1 ] || { echo "Unexpected exit $result: $specification" >&2; exit 1; }
    fi
    {
        echo "inject=$specification; exit=1; no timeout"
        cat "report/evidence/fault-$label.err"
    } >> report/evidence/faults.txt
done
# Allocation failure in getline: 32 MiB virtual-memory limit, larger input stream.
if awk 'BEGIN { for (i=0; i<50 000 000; ++i) printf "a"; print "" }' 2>/dev/null | \
    (ulimit -v 32 768; timeout 15 ./build-linux/parent ./build-linux/child1 ./build-linux/child2) \
    > report/evidence/memory-error.out 2> report/evidence/memory-error.err; then
    echo 'Memory-limit test unexpectedly succeeded' >&2
    exit 1
else
    result=$?
    [ "$result" -eq 1 ] || { echo "Memory-limit test exited $result" >&2; exit 1; }
fi

```

```
grep -q 'Cannot allocate memory' report/evidence/memory-error.err
{
    echo 'getline allocation failure: virtual-memory limit 32 768 KiB; exit=1; no timeout'
    cat report/evidence/memory-error.err
} >> report/evidence/faults.txt
echo 'Fault-injection and memory-limit tests passed.'
```

Листинг A.10 — Файл src/tools/Dockerfile

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential cmake strace ca-certificates \
    && apt-get clean
WORKDIR /lab
CMD ["bash"]
```

Листинг Б.1 — Полный журнал демонстрационного запуска

21

```

170 <... dup3 resumed>)          = 0
168 <... close resumed>)        = 0
169 <... dup3 resumed>)          = 1
168 close(8 <unfinished ...>
170 dup3(8, 1, 0 <unfinished ...>
168 <... close resumed>)          = 0
169 close(3 <unfinished ...>
168 fcntl(4, F_GETFL <unfinished ...>
170 <... dup3 resumed>)          = 1
168 <... fcntl resumed>)          = 0x1 (flags O_WRONLY)
169 <... close resumed>)          = 0
168 fcntl(4, F_SETFL, O_WRONLY|O_NONBLOCK <unfinished ...>
170 close(3 <unfinished ...>
168 <... fcntl resumed>)          = 0
169 close(4 <unfinished ...>
168 fcntl(7, F_GETFL <unfinished ...>
170 <... close resumed>)          = 0
168 <... fcntl resumed>)          = 0 (flags O_RDONLY)
169 <... close resumed>)          = 0
168 fcntl(7, F_SETFL, O_RDONLY|O_NONBLOCK <unfinished ...>
170 close(4 <unfinished ...>
168 <... fcntl resumed>)          = 0
169 close(5 <unfinished ...>
168 getpid( <unfinished ...>
170 <... close resumed>)          = 0
168 <... getpid resumed>)          = 168
169 <... close resumed>)          = 0
168 write(2, "[parent pid=168, child1=169, child2=170] started\n", 49 <unfinished ...>
170 close(5 <unfinished ...>
169 close(6 <unfinished ...>
168 <... write resumed>)          = 49
170 <... close resumed>)          = 0
168 write(2, "Enter lines. Press Ctrl-D to finish.\n", 37 <unfinished ...>
169 <... close resumed>)          = 0
168 <... write resumed>)          = 37
170 close(6 <unfinished ...>
168 getrandom( <unfinished ...>
169 close(7 <unfinished ...>
168 <... getrandom resumed>"\x35\xf2\xf9\x5c\x63\x1d", 8, GRND_NONBLOCK) = 8
170 <... close resumed>)          = 0
168 brk(NULL <unfinished ...>
169 <... close resumed>)          = 0
168 <... brk resumed>)            = 0xaaaaecb1e000
170 close(7 <unfinished ...>
168 brk(0xaaaaecb3f000 <unfinished ...>
169 close(8 <unfinished ...>
168 <... brk resumed>)            = 0xaaaaecb3f000
170 <... close resumed>)          = 0
168 fstat(0, <unfinished ...>
169 <... close resumed>)          = 0
170 close(8 <unfinished ...>
168 <... fstat resumed>{st_mode=S_IFREG|0644, st_size=25, ...}) = 0
169 execve("/build-linux/child1", ["/build-linux/child1"], 0xffffffff234328 /* 5 vars */ <unfinished ...>
168 read(0, <unfinished ...>
170 <... close resumed>)          = 0
168 <... read resumed>"hello world\n a b \n", 4096) = 25
168 ppoll([{fd=4, events=POLLOUT}, {fd=7, events=POLLIN}], 2, NULL, NULL, 0 <unfinished ...>
170 execve("/build-linux/child2", ["/build-linux/child2"], 0xffffffff234328 /* 5 vars */ <unfinished ...>
168 <... ppoll resumed>)          = 1 ([{fd=4, revents=POLLOUT}])
168 write(4, "hello world\n", 14) = 14
169 <... execve resumed>)          = 0
168 ppoll([{fd=4, events=0}, {fd=7, events=POLLIN}], 2, NULL, NULL, 0 <unfinished ...>

```

[illegible]

```

170 munmap(0xffffac4fa000, 24576 <unfinished ...>
169 munmap(0xffffa887e000, 8192 <unfinished ...>
170 <... munmap resumed>) = 0
169 <... munmap resumed>) = 0
170 munmap(0xffffac6be000, 40848 <unfinished ...>
169 munmap(0xffffa8a3e000, 57232 <unfinished ...>
170 <... munmap resumed>) = 0
169 <... munmap resumed>) = 0
170 mprotect(0xffffac69a000, 77824, PROT_NONE <unfinished ...>
169 mprotect(0xffffa8a1a000, 77824, PROT_NONE <unfinished ...>
170 <... mprotect resumed>) = 0
169 <... mprotect resumed>) = 0
170 mmap(0xffffac6ad000, 20480, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x19d000 <unfinished ...>
169 mmap(0xffffa8a2d000, 20480, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x19d000 <unfinished ...>
170 <... mmap resumed>) = 0xffffac6ad000
169 <... mmap resumed>) = 0xffffa8a2d000
170 mmap(0xffffac6b2000, 49040, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1,
0 <unfinished ...>
169 mmap(0xffffa8a32000, 49040, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1,
0 <unfinished ...>
170 <... mmap resumed>) = 0xffffac6b2000
169 <... mmap resumed>) = 0xffffa8a32000
170 close(3 <unfinished ...>
169 close(3 <unfinished ...>
170 <... close resumed>) = 0
169 <... close resumed>) = 0
170 set_tid_address(0xffffac700fb0 <unfinished ...>
169 set_tid_address(0xffffa8a84fb0 <unfinished ...>
170 <... set_tid_address resumed>) = 170
169 <... set_tid_address resumed>) = 169
170 set_robust_list(0xffffac700fc0, 24 <unfinished ...>
169 set_robust_list(0xffffa8a84fc0, 24 <unfinished ...>
170 <... set_robust_list resumed>) = 0
169 <... set_robust_list resumed>) = 0
170 rseq(0xffffac701600, 0x20, 0, 0xd428bc00 <unfinished ...>
169 rseq(0xffffa8a85600, 0x20, 0, 0xd428bc00 <unfinished ...>
170 <... rseq resumed>) = 0
169 <... rseq resumed>) = 0
170 mprotect(0xffffac6ad000, 12288, PROT_READ <unfinished ...>
169 mprotect(0xffffa8a2d000, 12288, PROT_READ <unfinished ...>
170 <... mprotect resumed>) = 0
169 <... mprotect resumed>) = 0
170 mprotect(0xaaaae4f2f000, 4096, PROT_READ <unfinished ...>
169 mprotect(0xaaaaade8f000, 4096, PROT_READ <unfinished ...>
170 <... mprotect resumed>) = 0
169 <... mprotect resumed>) = 0
170 mprotect(0xffffac706000, 8192, PROT_READ <unfinished ...>
169 mprotect(0xffffa8a8a000, 8192, PROT_READ <unfinished ...>
170 <... mprotect resumed>) = 0
169 <... mprotect resumed>) = 0
170 prlimit64(0, RLIMIT_STACK, NULL, <unfinished ...>
169 prlimit64(0, RLIMIT_STACK, NULL, <unfinished ...>
170 <... prlimit64 resumed>{rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
169 <... prlimit64 resumed>{rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
170 munmap(0xffffac6fd000, 8935 <unfinished ...>
169 munmap(0xffffa8a81000, 8935 <unfinished ...>
170 <... munmap resumed>) = 0
169 <... munmap resumed>) = 0
170 getpid( <unfinished ...>
169 getpid( <unfinished ...>

```

```

170 <... getpid resumed>          = 170
169 <... getpid resumed>          = 169
170 write(2, "[child2 pid=170] started\n", 25 <unfinished ...>
169 write(2, "[child1 pid=169] started\n", 25 <unfinished ...>
170 <... write resumed>           = 25
170 getrandom( <unfinished ...>
169 <... write resumed>           = 25
170 <... getrandom resumed> "\xc0\xab\xfb\x19\x32\x55\x6f", 8, GRND_NONBLOCK) = 8
169 getrandom( <unfinished ...>
170 brk(NULL <unfinished ...>
169 <... getrandom resumed> "\x6d\x1e\xcf\x66\xa5\xd9\x2b\xc0", 8, GRND_NONBLOCK) = 8
170 <... brk resumed>             = 0xaaaae77e5000
169 brk(NULL <unfinished ...>
170 brk(0xaaaae7806000 <unfinished ...>
169 <... brk resumed>             = 0xaaaaada1e9000
170 <... brk resumed>             = 0xaaaae7806000
169 brk(0xaaaaada20a000 <unfinished ...>
170 fstat(0, <unfinished ...>
169 <... brk resumed>             = 0xaaaaada20a000
170 <... fstat resumed> {st_mode=S_IFIFO|0600, st_size=0, ...} = 0
169 fstat(0, <unfinished ...>
170 read(0, <unfinished ...>
169 <... fstat resumed> {st_mode=S_IFIFO|0600, st_size=0, ...} = 0
169 read(0, "hello world\n", 4096) = 14
169 write(1, "HELLO WORLD\n", 14) = 14
170 <... read resumed> "HELLO WORLD\n", 4096) = 14
169 read(0, <unfinished ...>
170 write(1, "HELLO WORLD\n", 12 <unfinished ...>
168 <... ppoll resumed>          = 1 ({fd=7, revents=POLLIN})
170 <... write resumed>          = 12
168 read(7, <unfinished ...>
170 read(0, <unfinished ...>
168 <... read resumed> "HELLO WORLD\n", 4096) = 12
168 write(1, "HELLO WORLD\n", 12) = 12
168 ppoll([{fd=4, events=POLLOUT}, {fd=7, events=POLLIN}], 2, NULL, NULL, 0) = 1 ({fd=4, revents=POLLOUT})
168 write(4, " a b \n", 11) = 11
169 <... read resumed> " a b \n", 4096) = 11
168 ppoll([{fd=4, events=0}, {fd=7, events=POLLIN}], 2, NULL, NULL, 0 <unfinished ...>
169 write(1, " A B \n", 11) = 11
170 <... read resumed> " A B \n", 4096) = 11
169 read(0, <unfinished ...>
170 write(1, " A B \n", 6 <unfinished ...>
168 <... ppoll resumed>          = 1 ({fd=7, revents=POLLIN})
170 <... write resumed>          = 6
168 read(7, <unfinished ...>
170 read(0, <unfinished ...>
168 <... read resumed> " A B \n", 4096) = 6
168 write(1, " A B \n", 6) = 6
168 read(0, "", 4096) = 0
168 close(4) = 0
169 <... read resumed> "", 4096) = 0
168 close(7 <unfinished ...>
169 close(0 <unfinished ...>
168 <... close resumed>          = 0
168 wait4(169, <unfinished ...>
169 <... close resumed>          = 0
169 close(1 <unfinished ...>
170 <... read resumed> "", 4096) = 0
169 <... close resumed>          = 0
170 close(0 <unfinished ...>
169 getpid( <unfinished ...>
170 <... close resumed>          = 0

```

```

169 <... getpid resumed>          = 169
170 close(1 <unfinished ...>)
169 write(2, "[child1 pid=169] finished\n", 26 <unfinished ...>)
170 <... close resumed>          = 0
170 getpid( <unfinished ...>)
169 <... write resumed>          = 26
170 <... getpid resumed>         = 170
169 exit_group(0 <unfinished ...>)
170 write(2, "[child2 pid=170] finished\n", 26 <unfinished ...>)
169 <... exit_group resumed>      = ?
170 <... write resumed>          = 26
170 exit_group(0)                = ?
169 +++ exited with 0 +++
168 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 169
168 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=169, si_uid=0, si_status=0, si_utime=0,
si_stime=0} ---
168 wait4(170, <unfinished ...>)
170 +++ exited with 0 +++
168 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 170
168 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=170, si_uid=0, si_status=0, si_utime=0,
si_stime=0} ---
168 getpid()                     = 168
168 write(2, "[parent pid=168] finished\n", 26) = 26
168 exit_group(0)                = ?
168 +++ exited with 0 +++

```